

Smart Stock Inventory Optimization for Retail Stores

1. Project Overview

This project focuses on creating a smart system that helps retail stores manage inventory efficiently. It predicts product demand, avoids overstock/understock, and provides suggestions for optimal stock levels.

2. Problem Statement

Retail stores struggle with inaccurate demand prediction, stockouts, and extra inventory. A data-driven system is needed to make stock planning easier and more accurate.

3. Objectives

- Predict future product demand
- Recommend reorder levels
- Reduce stock shortages and excess
- Provide simple insights for store managers

4. System Structure

A. Frontend

- Displays stock levels, alerts, and recommendations
- Simple dashboard for managers

B. Backend

- Processes sales data
- Runs forecasting logic
- Generates stock recommendations

C. Dataset

- Product list
- Historical sales data
- Daily stock updates

5. Python Concept for Day-1: f-Strings

What is an f-string?

A simple way to format strings using `{}` inside a string that starts with `f`.

Example:

```
product = "Milk"  
stock = 15  
print(f"{product} has only {stock} units left.")
```

Example (related to project):

```
required = 40  
current = 25  
print(f"Low stock alert: Need {required}, but only {current} available.")
```

Day 2 – Git & GitHub Notes**1. Initialize a Git Repository**

Start a new Git repository:

```
git init
```

2. Check Repository Status

View what has changed, staged, or unstaged:

```
git status
```

3. Add Files to Staging Area

Add all files:

```
git add
```

Add a specific file:

```
git add filename
```

4. Remove Files from Staging

If you staged something by mistake:

```
git restore --staged filename
```

5. Commit Changes

Save your staged work:

```
git commit -m "Initial commit"
```

Amend last commit message:

```
git commit --amend -m "New message"
```

6. View Commit History

git log

Compact one-line log:

git log --oneline

7. Create a New GitHub Repository

1. Open GitHub → Click **New Repository**
2. Enter repository name
3. Leave **README unchecked** if pushing an existing project
4. Click **Create Repository**

8. Connect Local Repo to GitHub Remote

Add remote origin:

git remote add origin https://github.com/username/repo-name.git

Verify remote:

git remote -v

Set main branch:

git branch -M main

9. Push Code to GitHub

First push:

git push -u origin main

Regular push:

git push

10. Pull Latest Changes from GitHub

git pull

Pull from a specific branch:

git pull origin main

11. Clone a Repository

git clone https://github.com/username/repo.git

12. Branching Commands

Create a new branch:

git branch feature-1

Switch branch:

git checkout feature-1

Create + switch:

git checkout -b feature-1

List branches:

git branch

Delete branch:

git branch -d branchname

13. Merging Branches

Merge a branch into main:

git checkout main

git merge feature-1

14. Stash Changes (Temporary Save)

Save unstaged work:

git stash

Show stashes:

git stash list

Apply latest stash:

git stash apply

15. Python Virtual Environment

A **Python Virtual Environment (venv)** is an isolated workspace that allows you to install packages for a specific project without affecting the global Python installation.

Why use a virtual environment?

- Avoid version conflicts between projects
- Keep project dependencies isolated
- Maintain cleaner and more manageable Python setups

Commands

Create a Virtual Environment

```
python -m venv env
```

What is an API?

An **API (Application Programming Interface)** allows two applications to communicate.

For example:

- Your frontend sends a request → “Give me user data”
- The backend API responds → Sends JSON data back

APIs use **HTTP methods**:

- **GET** → Take data
- **POST** → Send data
- **PUT** → Update data
- **DELETE** → Remove data

APIs act like **bridges between systems**.

Flask – What It Is & How to Use It

Flask is a small and simple Python framework used to build web APIs.

How Flask works (simple explanation):

- You create **routes** like /login, /users, /product.
- Each route responds to requests (GET, POST, etc.).
- Flask sends back **JSON response** to the client.

When to use Flask?

- Small projects
- Simple APIs
- When you need flexibility
- When learning backend basics

Flask is lightweight and very beginner-friendly.

FastAPI – What It Is & How to Use It

FastAPI is a modern, very fast API framework for Python.

Why is FastAPI popular?

- Extremely fast performance

- Automatic documentation (*Swagger UI*)
- Easy to validate data
- Great for modern applications

When to use FastAPI?

- Real-world production APIs
- Large applications
- High-performance requirements
- Projects needing built-in docs

FastAPI is currently one of the most used Python API frameworks.

Postman – What It Is & How to Use It

Postman is a tool for testing APIs.

What you can do with Postman:

- Send **GET, POST, PUT, DELETE** requests
- Add **JSON body**
- Add **headers** like authentication tokens
- Test if your API is working
- Save all your API calls in collections
- View responses clearly

Example of how you use Postman:

1. Type the API URL
2. Select the method (GET/POST/etc.)
3. If POST → add body (JSON)
4. Hit **Send**
5. Check the response

It helps developers test backend APIs without writing a frontend.

Day 4 – RDBMS, CRUD, and Non-RDBMS

1. What is RDBMS?

RDBMS (Relational Database Management System) is a type of database that stores data in **table format** with rows and columns.

Examples:

- **MySQL**
- **PostgreSQL**
- **Oracle**
- **SQL Server**
- **SQLite**

Features of RDBMS:

- Data stored in **tables**
- Tables can be **linked using relationships**
- Uses **SQL (Structured Query Language)**
- Ensures **data consistency, ACID properties, and data integrity**

When to use RDBMS:

- Banking systems
- E-commerce
- Ticket booking apps
- Any platform requiring structured, consistent data

2. What is CRUD?

CRUD represents the **four basic operations** performed on database data.

Operation	Meaning	Purpose
C – Create	Add new data	Insert new user/product/etc.
R – Read	Retrieve data	View user list, search items
U – Update	Modify existing data	Change password, edit profile
D – Delete	Remove data	Delete account, remove item

CRUD is used in **all database systems** — RDBMS and NoSQL alike.

4. Normalization – What It Is?

Normalization is a process in databases to remove **redundant data** and improve **data integrity**.

Why normalization?

- Reduces duplicate data
- Makes database clean & organized
- Avoids update anomalies
- Ensures relationships are properly structured

5. Normal Forms (NFs)

These are rules/standards used to normalize data.

1NF – First Normal Form

A table is in 1NF if:

- All values are **atomic** (no multiple values in one cell)
- No repeating groups or lists
- Each column has a unique name
- Data is stored in rows & columns

Example issue:

A column containing:

Phones: 9876543210, 9123456789 → Not allowed in 1NF.

2NF – Second Normal Form

A table is in 2NF if:

- It is already in **1NF**
- **No partial dependency** (no non-key column depends on part of a composite key)

This applies only when:

A table has composite primary key

If a table has single primary key, 2NF = 1NF

Goal: Remove columns that depend only on one part of a composite key.

3NF – Third Normal Form

A table is in 3NF if:

- It is already in **2NF**

- **No transitive dependency**
(i.e., non-key columns should not depend on other non-key columns)

Example:

If *City* → *Pincode* → *State* is inside one table, it causes dependency problems.

Solution → Move to separate tables

3. What is Non-RDBMS (NoSQL)?

Non-RDBMS (NoSQL databases) do NOT store data in tables.

They store data in more flexible formats like:

- **Documents**
- **Key-Value pairs**
- **Graphs**
- **Wide-column stores**

Examples:

- **MongoDB** (Document database)
- **Firebase** (Cloud NoSQL)
- **Cassandra**
- **Redis** (Key-value store)

Features of NoSQL:

- Flexible structure (no fixed schema)
- Scales easily (horizontal scaling)
- Fast for large data and real-time applications

When to use NoSQL:

- Social media applications
- Analytics platforms
- Real-time apps (chat apps, notifications)
- Big data
- When structure changes frequently

Day 5 – Machine Learning Types: Supervised, Unsupervised & Reinforcement Learning

1. What is Machine Learning?

Machine Learning (ML) is a field of AI where systems learn patterns from data and make predictions/decisions **without being explicitly programmed**.

ML is mainly divided into three types:

1. **Supervised Learning**
2. **Unsupervised Learning**
3. **Reinforcement Learning**

2. Supervised Learning

Supervised learning uses **labeled data**.

You teach the model by showing both **input** and **correct output**.

Example of labeled data:

Input (Hours studied)	Output (Score)
-----------------------	----------------

2	50
---	----

4	70
---	----

6	90
---	----

The model learns the pattern and predicts future outputs.

Used for:

- Prediction
- Classification
- Regression

Real-world examples:

- Spam vs Not Spam (classification)
- Predicting house prices (regression)
- Face recognition
- Disease prediction

Algorithms:

- Linear Regression

- Logistic Regression
- Decision Trees
- SVM
- Random Forest
- Neural Networks

3. Unsupervised Learning

Unsupervised learning uses **unlabeled data**.

There is **no output given** — the model finds hidden patterns on its own.

Example of unlabeled data:

Age, income, spending score (but no “category” given)

The model will group similar behaviours.

Used for:

- Clustering
- Pattern detection
- Grouping customers
- Anomaly detection

Real-world examples:

- Customer segmentation
- Market basket analysis
- Grouping similar photos
- Fraud detection

Algorithms:

- K-Means
- Hierarchical Clustering
- PCA (Dimensionality Reduction)
- DBSCAN

4. Reinforcement Learning

Reinforcement learning is based on **learning by trial and error**.

Here:

- An **agent** interacts with an **environment**
- Performs **actions**
- Receives **rewards** or **penalties**
- Learns the best strategy over time

Real-world examples:

- Self-driving cars
- Robots learning tasks
- Game-playing AI (Chess, Go)
- Recommendation systems that adapt to user behaviour

Core concepts:

- **Agent** → Learner
- **Environment** → World where agent acts
- **Action** → What the agent does
- **Reward** → Positive/negative feedback
- **Policy** → Strategy learned

Artificial Intelligence (AI)

Definition

AI is a technology that enables machines to think and act like humans.

It allows computers to make decisions, learn from data, and solve problems.

Simple Real-Life Examples

- Google Maps suggesting fastest route
- Face unlock system recognizing your face
- ChatGPT answering questions like a human
- Self-driving cars making driving decision

AI = Making machines intelligent so they can perform tasks automatically.

Natural Language Processing (NLP)

Definition

NLP is a part of AI that helps computers understand, interpret, and respond to human language (text or speech).

Simple Real-Life Examples

- Siri / Alexa understanding voice commands
- Google Translate converting language
- Gmail identifying spam messages
- Sentiment analysis on product reviews (positive/negative)

NLP = Teaching machines to understand human language like English, Telugu, Hindi etc.

Deep Learning (DL)

Definition

Deep Learning is a part of Machine Learning that uses Neural Networks to learn patterns from large amounts of data.

Simple Real-Life Examples

- Instagram recognizing faces in photos
- YouTube recommending videos based on history
- Hospitals predicting diseases from X-ray or MRI images
- ChatGPT generating responses intelligently

Deep Learning = Brain-like learning, learning from big data and improving automatically.

Reinforcement Learning (RL)

Definition

Reinforcement Learning is a learning method where an agent learns by performing actions and receiving rewards or penalties. It learns through trial and error.

Simple Real-Life Examples

- Robot learning to walk by receiving reward for correct steps
- AlphaGo game AI defeating world champions
- Self-driving car learning to stop at signals
- Ad recommendation improving based on what users click

RL = Learn by doing, learn from mistakes, get better step-by-step (like training a pet dog using rewards).

LSTM (Long Short-Term Memory)

Definition

LSTM is a special type of RNN (Recurrent Neural Network) that can remember information for a long time. It is used for sequence-related data like sentences, speech, video, time-series.

Normal RNNs forget old information when the sequence becomes long, but LSTM can store important information and remove unnecessary information.

Why LSTM is used?

Because normal RNNs forget old context.

Example sentence:

“I live in India. India is a beautiful country.”

To understand the second word India, the model must remember India from the first sentence.

LSTM can remember it — RNN can't.

How LSTM Works

LSTM has three gates that help manage memory:

Forget Gate

- Removes unwanted or unnecessary information
(Deletes things that are not important)

Input Gate

- Adds and saves important new information
(Stores new useful details)

Output Gate

- Gives important information as output
(Provides final useful result to the next step)

Example

Sentence: “*The movie was very interesting and the actors were amazing.*”

- Forget gate removes useless words like "the"

- Input gate stores useful words like "interesting", "amazing"
- Output gate gives final meaning → Positive sentiment

Model result → Review is Positive

Where LSTM is used in Real Life

- Google Translate understanding long sentences
- Siri / Alexa voice assistants
- Stock price prediction
- Weather forecasting
- ChatGPT context understanding
- Music/video recommendation systems
- Detecting emotions from text

Sentiment Analysis

Definition

Sentiment Analysis is a technique in **NLP (Natural Language Processing)** where a computer **understands emotions or opinions** from text and classifies them as **positive, negative, or neutral**.

It is used to identify what people feel about a product, service, movie, or event.

What Sentiment Analysis Does

It reads text like:

- reviews
- comments
- tweets
- feedback

- messages

And decides the **emotion** behind the sentence.

Simple Examples

Sentence | Detected Sentiment

“I love this phone, it works perfectly!” → Positive

“This product is very bad and I hate it.” → Negative

“The movie was okay, not too good or bad.” → Neutral

Tokenization

Definition

Tokenization is the process of breaking a sentence or text into smaller parts called **tokens**.

These tokens can be **words**, **subwords**, **characters**, or **sentences**.

Why we use Tokenization?

Because computers cannot directly understand long sentences—so we split text into small understandable pieces for analysis.

Example

Input sentence:

I love learning Artificial Intelligence.

After tokenization (word-level):

["I", "love", "learning", "Artificial",
"Intelligence"]

Sentence Tokenization:

["I love learning AI.", "It is interesting."]

Real-life example

Lemmatization

Definition

Lemmatization reduces a word to its **base / dictionary form (lemma)** by considering the **context and part of speech (POS)**.

Why we use Lemmatization?

To treat different forms of a word as the same word.

Example: *running*, *ran*, and *runs* should be understood as **run**.

Example

Before Lemmatization → After Lemmatization:

running → *run*

studies → *study*

better → *good*

eaten → *eat*

Real-life example

In **sentiment analysis**, reducing all word forms to a base helps better understand meaning:

"I am running fast"

"I run every day"

When you use Google search, Google breaks your query into tokens to understand key words and provide relevant results.

Support Vector Machine (SVM)

Definition

SVM (Support Vector Machine) is a **supervised machine learning algorithm** used for **classification** and **regression**, but mostly used for **classification**.

SVM tries to find the **best separating line (or boundary)** called **hyperplane** that separates different classes of data.

Simple Explanation

Imagine we have two categories of points:

- **Red points**
- **Blue points**

SVM tries to draw a line between them **so that the distance between the line and the nearest points is maximum**.

These nearest points are called **Support Vectors**.

So the main goal of SVM:

Find the best line that separates the classes clearly with maximum margin.

Key Concepts in SVM

1. Hyperplane

The **boundary** that separates data into classes.

- In 2D → it is a **line**
- In 3D → it is a **plane**
- In high dimension → it is a **hyperplane**

2. Margin

The **distance** between the hyperplane and the closest data points.
SVM tries to **maximize this margin**.

3. Support Vectors

The **data points closest to the boundary** which decide the position of the hyperplane.

Types of SVM

1. Hard Margin SVM

Used when data is **perfectly separable** — no overlapping.

- Example: Clear separation of male and female height groups with no mixing.

2. Soft Margin SVM

Allows some misclassification when data is **not perfectly separable** — realistic situations.

- Example: Classifying emails into spam or not spam; some messages may be confusing.

AI Agents

What is an AI Agent?

An **AI Agent** is a smart system that can **observe**, **think**, and **take action** in order to achieve a goal.

In simple words:

An AI Agent understands the environment, makes decisions, and performs tasks automatically like a human.

How AI Agents Work

AI agents follow a continuous cycle:

- They **observe** the environment using sensors.
- They **decide** the best action based on their logic or learning.
- They **act** using actuators or output devices.
- They **learn** from the results and improve.

For example:

A **self-driving car** sees the road, decides where to move, and controls the steering.

Main Components of an AI Agent

- **Sensors** – collect information (example: camera, GPS, microphone, temperature sensor)
- **Actuators** – perform actions (example: wheels, robotic arms, speaker)

- **Agent Program** – decision-making logic or algorithm
- **Environment** – place where the agent works (road, home, website, hospital, game)

Types of AI Agents (Simple & with Examples)

1. Simple Reflex Agent

Works only based on current inputs, without remembering past.

Example:

A fan that turns ON automatically when the temperature is high and OFF when it is low.

2. Model-Based Reflex Agent

Uses memory of past actions to decide better.

Example:

A robotic vacuum cleaner remembers which areas are already cleaned.

3. Goal-Based Agent

Decides actions based on a goal to achieve.

Example:

Google Maps trying to find the best route to reach a destination.

4. Utility-Based Agent

Chooses the best option that provides the highest satisfaction or benefit.

Example:

Netflix recommending the best movies you might like.

5. Learning Agent

Learns from experience and improves performance automatically.

Example:

ChatGPT improving responses over time, or Siri learning your voice commands.

How to Create an API Key in OpenAI

Step 1: Visit the OpenAI Website

Open a browser and go to the OpenAI account page:

<https://platform.openai.com/>

Step 2: Sign Up or Log In

If you already have an account, log in with email / Google / Microsoft.

Step 3: Open the Dashboard

After logging in, you will see the OpenAI dashboard.

Step 4: Go to API Keys Section

On the left side menu, click **"API Keys"** or **"View API Keys"**.

Step 5: Create a New API Key

Click **"Create new secret key"** button.

Step 6: Copy the Key

You will see something like:

`sk-abcdefghijklmnopqrstuvwxyz...`

Step 7: Store It Safely

Save it in a secure place (never post it publicly).

Put it in a `.env` file or environment variable in projects.

Example `.env` file:

`OPENAI_API_KEY=sk-abcdefghijklmnopqrstuvwxyz`

How to Create a Chatbot

Step 1: Decide Purpose of Chatbot

Choose what type of chatbot you want:

- Customer support bot
- College admission enquiry bot
- Food ordering bot
- Personal AI assistant
- FAQ bot

Step 2: Choose a Technology

You can create chatbot using:

- Python + OpenAI API
- Dialogflow (Google)
- Rasa (framework)
- No-code platforms (ManyChat, Botpress)

creating Python chatbot using OpenAI.

Step 3: Install Required Package

Open terminal and type:

```
pip install openai
```

Step 4: Write the Chatbot Program

Here's a simple example:

```
from openai import OpenAI

client = OpenAI(api_key="your_api_key")

while True:

    user = input("You: ")
```



```
        if user.lower() == "exit":  
            break  
  
        response = client.chat.completions.create(  
            model="gpt-4o-mini",  
            messages=[{"role": "user", "content":  
user}]  
        )  
  
        print("Bot:",  
response.choices[0].message["content"])
```

Step 5: Run the Program

Run the python file:

```
python chatbot.py
```

How Chatbots Work (Simple)

- Understand the user message using **NLP**
- Process text using **ML or AI model**
- Generate reply automatically
- Learn from feedback and improve

Where Chatbots Can Be Used

- Websites (Helpdesk / FAQs)

- WhatsApp & Telegram bots
- Voice assistants
- Online shopping customer support
- University enquiry systems
- Banking and healthcare support

Logistic Regression

Definition

Logistic Regression is a classification algorithm used to predict a category or class (Yes/No, True/False, 0/1).

It uses the Sigmoid Function to convert values into probabilities between 0 and 1.

Where it is used

- Email Spam Detection (Spam / Not Spam)
- Predicting if a student will pass or fail based on study hours
- Medical diagnosis (Disease / No disease)
- Fraud Detection (Fraud / Genuine)

Example

If a student studies 4 hours daily, logistic regression predicts the chance of passing.

If probability $> 0.5 \rightarrow$ Pass

If probability $< 0.5 \rightarrow$ Fail

It does not predict numbers; it predicts classes.

KNN (K-Nearest Neighbors)

Definition

KNN is a classification algorithm based on similarity.
It classifies something based on its nearest neighbors.

How it works

1. Choose the value of K (example $K=3$)
2. Find the nearest 3 points around the new data
3. The majority class becomes the result

Real-Life Example

Imagine you want to classify a new fruit based on sweetness and size.

Look at its nearest fruits:

- 2 Apples
- 1 Orange

Since most neighbors are apples $\rightarrow$ the new fruit is apple.

Use cases

- Face recognition
- Movie recommendation (similar users)
- Pattern recognition in images

K-Means Clustering

Definition

K-Means is unsupervised learning used for clustering. It groups similar items into clusters without labels.

How it works

1. Choose number of clusters K
2. Place K random points called centroids
3. Assign each data point to nearest centroid
4. Recalculate centroid positions until stable

Real-Life Example A shopping mall groups customers based on purchasing behavior:

- Cluster 1: students buying low-budget products
- Cluster 2: families buying home items
- Cluster 3: wealthy customers buying luxury items

The algorithm discovered these groups without knowing categories earlier.

Uses

- Customer segmentation in marketing
- Grouping documents with similar topics
- Brain image segmentation in healthcare

Random Forest

Definition

Random Forest is a supervised machine learning algorithm used for classification and regression.

It builds multiple decision trees and combines their outputs to get a more accurate result.

Simple Idea

Instead of trusting one tree, use a forest of trees and take majority vote.

Example

Predict whether a loan applicant is risky or safe using features like salary, age, credit score.

Each tree gives a result:

- **Tree 1: Safe**
- **Tree 2: Safe**
- **Tree 3: Risky**
- **Tree 4: Safe**

Majority vote = Safe

Advantages

- Works well with large datasets
- Handles missing values
- Reduces overfitting
- Very accurate

Real-Time Uses

- Loan approval systems
- Disease prediction
- Stock market prediction
- E-commerce recommendation

SQL (Structured Query Language) is used to **manage and manipulate relational databases**.

Data Retrieval: SELECT

Filtering: WHERE, AND, OR, NOT, LIKE, IN, BETWEEN

Sorting: ORDER BY

Grouping & Aggregation: GROUP BY, HAVING

Joins: INNER, LEFT, RIGHT, FULL, CROSS, SELF

Combining Results: UNION, UNION ALL

Filtering Data – WHERE, AND, OR, NOT

```
SELECT * FROM Students WHERE Course = 'CSE' AND  
Marks > 70;
```

AND: both conditions must be true

OR: any condition can be true

NOT: reverses the condition

Pattern Matching – LIKE Operator

Patt ern	Meaning	Example
-------------	---------	---------

a%	Starts with 'a'	'Aaron', 'Arjun'
----	-----------------	---------------------

```
%a      Ends with 'a'      'Sita',
                                'Priya'
```

%or% Contains 'or' 'Saroja'

```
_r% Second letter 'Priya'
'r'
```

a_% Starts with 'a', 'Arjun',
≥2 characters 'Aaron'

```
a_-% Starts with 'a', 'Arjun',
    ≥3 characters 'Aaron'
```

```
a%0    Starts with 'a'  'a00',
        and ends with  'ap40'
        '0'
```

```
SELECT * FROM Employees
```

```
WHERE Name LIKE 'a%';
```

Sorting Data – ORDER BY

```
SELECT * FROM Students
```

ORDER BY Marks DESC;

- ASC = Ascending (default)
- DESC = Descending

Sorting by multiple columns:

ORDER BY Course ASC, Marks DESC;

4. Grouping Data – GROUP BY and HAVING

Example Table: Sales

```
SELECT SalesPerson, SUM(Amount) AS TotalSales  
FROM Sales  
GROUP BY SalesPerson  
HAVING SUM(Amount) > 30000;
```

- GROUP BY: groups rows with the same values
- HAVING: filters groups after aggregation
- Aggregate functions: **SUM()**, **COUNT()**, **AVG()**, **MAX()**, **MIN()**

5. Joins – Combining Tables

Join Type	Description	Example
INNER JOIN	Only matching rows	<code>SELECT * FROM Students INNER JOIN Courses ON Students.CourseID=Courses.CourseID;</code>
LEFT JOIN	All left table rows + matches	<code>LEFT JOIN</code>
RIGHT JOIN	All right table rows + matches	<code>RIGHT JOIN</code>
FULL JOIN	All rows from both tables	<code>FULL OUTER JOIN</code>

CROSS JOIN Cartesian product **CROSS JOIN**

SELF JOIN Table joined with itself **SELF JOIN**

6. Combining Queries – UNION vs UNION ALL

SELECT Name FROM TableA

UNION

SELECT Name FROM TableB; -- Removes duplicates

SELECT Name FROM TableA

UNION ALL

SELECT Name FROM TableB; -- Keeps duplicates

- **UNION** = unique rows only
- **UNION ALL** = includes duplicates

PYTHON -STRING CONCEPTS

open() → Opens a file.

```
f = open("demo.txt", "w") # open file in write mode
```

write() → Writes text into the file.

```
f.write("Hello World")
```

writelines() → Writes multiple lines at once.

```
f.writelines(["Line1\n", "Line2\n"])
```

read() → Reads the entire file.

```
print(f.readline())
```

readlines() → Reads all lines into a list.

```
lines = f.readlines()
```

close() → Closes the file.

```
f.close()
```

len() → Length of object.

```
len("Python") # 6
```

range() → Sequence of numbers.

```
list(range(5)) # [0,1,2,3,4]
```

print() → Displays output.

```
print("Hello")
```

type() → Shows data type.

```
type(10) # <class 'int'>
```

id() → Unique memory ID.

```
id(10)
```

sorted() → Returns sorted list.

```
sorted([3,1,2]) # [1,2,3]
```

enumerate() → Adds index to items.

```
for i,v in enumerate(["a","b"]):  
    print(i,v)
```

zip() → Combines iterables.

```
list(zip([1,2],['a','b'])) # [(1,'a'),(2,'b')]
```

Set methods

add() → Adds element.

```
s = {1,2}; s.add(3)
```

remove() → Removes element (error if not present).

```
s.remove(2)
```

discard() → Removes if present (no error).

```
s.discard(5)
```

pop() → Removes and returns random element.

```
s.pop()
```

clear() → Removes all elements.

```
s.clear()
```

union() → Combines sets.

```
{1,2}.union({2,3}) # {1,2,3}
```

intersection() → Common elements.

```
{1,2}.intersection({2,3}) # {2}
```

difference() → Elements not in other set.

```
{1,2,3}.difference({2}) # {1,3}
```

String Methods

strip() → Removes spaces.

```
" hello ".strip() # "hello"
```

split() → Splits into list.

```
"a,b,c".split(",") # ['a','b','c']
```

join() → Joins list into string.

```
", ".join(['a','b']) # "a,b"
```

replace() → Replace substring.

```
"hi world".replace("hi","hello")
```

upper()/lower() → Case change.

```
"abc".upper() # "ABC"
```

startswith()/endswith() → Check prefix/suffix.

```
"python".startswith("py") # True
```

find() → Position of substring.

```
"hello".find("lo") # 3
```

isdigit()/isalpha() → Check type.

"123".isdigit() # True

"abc".isalpha() # True

List Methods

append() → Add one item.

l=[1]; l.append(2)

extend() → Add multiple items.

l.extend([3,4])

insert() → Insert at position

l.insert(1, "x")

remove() → Remove first match.

l.remove("x")

pop() → Remove by index.l.pop(0)

index() → Position of element.

l.index(2)

count() → Count occurrences.

l.count(2)

sort() → Sort list.

`l.sort()`

Dictionary Methods

get() → Get value safely.

`d={"a":1}; d.get("a")`

keys()/values()/items() → Access keys, values, pairs.

`d.keys(); d.values(); d.items()`

update() → Add/modify entries.

`d.update({"b":2})`

pop() → Remove by key.

`d.pop("a")`

popitem() → Remove last pair.

`d.popitem()`

clear() → Empty dictionary.

`d.clear()`

Mathematical Functions

`abs(x)` → Returns the absolute value.

```
print(abs(-10)) # 10
```

`sum(iterable)` → Sums all items in an iterable.

```
print(sum([1,2,3])) # 6
```

`min(iterable)` → Returns the smallest element.

```
print(min([5,2,8])) # 2
```

`max(iterable)` → Returns the largest element.

```
print(max([5,2,8])) # 8
```

`pow(x, y)` → Raises `x` to the power `y`.

```
print(pow(2,3)) # 8
```

`round(x, n)` → Rounds to `n` decimal places.

```
print(round(3.14159,2)) # 3.14
```

Functional Programming Tools

`filter(func, iterable)` → Filters elements based on condition.

```
nums = [1,2,3,4,5]
```

```
evens = list(filter(lambda x: x%2==0, nums))
```

```
print(evens) # [2,4]
```

`map(func, iterable)` → Applies function to each element

```
nums = [1,2,3]
doubled = list(map(lambda x: x*2, nums))
print(doubled)  # [2,4,6]
```

reduce(func, iterable) → Reduces iterable to single value.

```
from functools import reduce
nums = [1,2,3,4]
product = reduce(lambda a,b: a*b, nums)
print(product)  # 24
```

Lambda Example

```
add_ten = lambda x: x+10
print(add_ten(5))  # 15
```

```
multiply = lambda a,b: a*b
print(multiply(3,4))  # 12
```

Input and Output

input() → Reads user input as string.

```
name = input("Enter your name: ")
print("Hello,", name)
```

format() → Formats strings.

```
age = 25
print("I am {} years old".format(age))
```

getattr(obj, attr) → Gets attribute value.

```
class Person:
    def __init__(self): self.name="Sahithya"
```

```
p = Person()
print(getattr(p,"name")) # Sahithya
```

`setattr(obj, attr, value)` → Sets attribute value.

```
setattr(p,"name","Copilot")
print(p.name) # Copilot
```

`hasattr(obj, attr)` → Checks if attribute exists.

```
print(hasattr(p,"name")) # True
```

`delattr(obj, attr)` → Deletes attribute.

```
delattr(p,"name")
print(hasattr(p,"name")) # False
```

`isinstance(obj, class)` → Checks if object is instance of class.

```
print(isinstance(p, Person)) # True
```

`issubclass(class1, class2)` → Checks subclass relationship

```
class Student(Person): pass
print(issubclass(Student, Person)) # True
```

Miscellaneous Functions

`globals()` → Returns dictionary of global variables.

```
print(globals().keys())
```

`locals()` → Returns dictionary of local variables.

```
def test():  
    x=10  
    print(locals())  
test()
```

`callable(obj)` → Checks if object can be called.

```
print(callable(print)) # True
```

`exec(str)` → Executes Python code from string.

```
exec("x=5; print(x)") # 5
```

`eval(str)` → Evaluates expression from string.

```
python
```

```
print(eval("2+3")) # 5
```

Exception Handling

```
try:  
    x = int("abc") # Error  
except ValueError as e:  
    print("Error occurred:", e)  
finally:
```

```
print("This block always runs")
```

Error occurred: invalid literal for int() with base 10: 'abc'

This block always runs

Stack & Queue

Stack

- Follows **LIFO** (Last In, First Out)
- Operations:
 - `push()` → add item
 - `pop()` → remove last item

Queue

- Follows **FIFO** (First In, First Out)

What are Decorators?

A **decorator** is a function that modifies another function's behavior **without changing its code**.

What are Closures?

A **closure** is a function that remembers variables from its **enclosing scope**, even after that scope has finished executing.

Function with Access to Enclosing Scope

This is exactly what a **closure** provides — inner functions can access variables from outer functions.

What is a Lambda Function?

A **lambda** is a small anonymous function written in one line.

```
square = lambda x: x * x
```

Multiple vs Multilevel Inheritance

A class inherits from **multiple parent classes**.

Multilevel Inheritance

A class inherits from a parent, which itself inherits from another parent

Monkey Patching

Changing class or module behavior **at runtime**.

Syntax Error

- Occurs **before execution**
- Code structure is invalid Example: missing colon

Runtime Error

- Occurs **during execution**
- Example: division by zero

Invalid Code Error vs Runtime Error

- **Invalid code error** → syntax/indentation mistakes
- **Runtime error** → code is valid but fails during execution

. pip

pip is Python's package manager used to install external libraries

```
pip install numpy
```

Delete File in Python

Python's Key Features

Interpreted

High-level

Dynamically typed

Object-oriented

Cross-platform

Supports multiple paradigms

Extensive standard library

Python's Data Types

Int, float, complex, str, list, tuple, set, dict, bool, NoneType

What is PEP 8?

PEP 8 is Python's **official style guide**. It ensures:

- Clean code
- Consistent formatting

- Better readability

Mutable

Can be changed after creation Examples: `list`, `dict`, `set`

Immutable

Cannot be changed Examples: `int`, `float`, `str`, `tuple`

Where is Python's Memory Managed?

- **Private Heap** → stores objects
- **Reference Counting** → tracks usage
- **Garbage Collector** → removes unused objects

Indentation in Python

Python uses indentation to define code blocks. Incorrect indentation

→ **SyntaxError**

How is Python Interpreted?

Python code is:

1. Compiled into **bytecode**
2. Executed by the **Python Virtual Machine (PVM)**

Software development models

Waterfall model

The Waterfall model is a linear and sequential approach to software development. The project is divided into distinct phases, and each phase must be completed before the next one begins. There is very little overlap between stages, and going back to a previous phase is difficult and costly.

Phases of the Waterfall model:

1. Requirement analysis: All system requirements are collected and documented in detail before any design or coding begins.
2. System design: The overall architecture, data structures, modules, and interfaces are designed based on the requirements.
3. Implementation (coding): Developers write the code according to the design specifications.
4. Integration and testing: Individual modules are integrated and then tested as a complete system to find and fix defects.
5. Deployment: The fully tested system is delivered to the customer or moved into the production environment.
6. Maintenance: Bug fixes, updates, and enhancements are performed after the system is in use.

Advantages of the Waterfall model:

- Simple and structured: Easy to understand and manage because of its clear, linear phases.
- Good for stable requirements: Works well when requirements are well-understood and unlikely to change.
- Clear documentation: Each phase produces documentation, which is useful for future maintenance and audits.

Disadvantages of the Waterfall model:

- Inflexible to change: Difficult to accommodate changing requirements once the project has moved past the early stages.

- Late testing and feedback: Problems are often found late in the process, during integration and testing.
- Risk of mismatch: The final product may not fully match the user's actual needs because users see it only at the end.

Agile model

The Agile model is an iterative and incremental approach to software development. Instead of delivering the entire product at once, work is broken into small cycles (often called sprints or iterations), and each cycle delivers a working part of the software. Requirements and solutions evolve through collaboration between cross-functional teams and stakeholders.

Key principles of Agile:

- Customer collaboration: Continuous interaction with the customer to refine requirements and priorities.
- Iterative development: The product is built in small increments, with each iteration including planning, design, coding, and testing.
- Embrace change: Changing requirements are welcomed, even late in development, if they add value.
- Frequent delivery: Working software is delivered frequently, giving early and regular feedback.
- Self-organizing teams: Teams manage their own work and collaborate closely to achieve goals.

Agile process (typical Scrum-style cycle):

1. Backlog creation: A prioritized list of features and requirements (product backlog) is created.
2. Sprint planning: The team selects a set of items from the backlog to complete within a short time-boxed period (usually 1–4 weeks).
3. Design and development: The selected features are designed and developed during the sprint.

4. Testing: Testing happens within the same sprint to ensure the increment is working and of good quality.
5. Review and demo: At the end of the sprint, the team demonstrates the working increment to stakeholders for feedback.
6. Retrospective: The team reflects on what went well, what didn't, and how to improve in the next sprint.

Advantages of the Agile model:

- Flexible and adaptive: Easily accommodates changes in requirements and priorities.
- Early value delivery: Users start getting usable features early in the project.
- Continuous feedback: Frequent feedback improves alignment with user needs.
- Better risk management: Issues are discovered and addressed in small cycles rather than at the end.

Disadvantages of the Agile model:

- Requires active customer involvement: Success depends on continuous participation from stakeholders.
- Less emphasis on extensive documentation: Can cause problems later if documentation is not maintained.
- Harder to predict: Scope, time, and cost can be harder to estimate upfront for the whole project.

Sorting Algorithms

1. Bubble Sort

- **Idea:** Compare neighbors and swap if out of order.
- **Example:** You have numbers [5, 3, 8, 4].
 - Compare 5 and 3 → swap → [3, 5, 8, 4].
 - Compare 5 and 8 → no swap.

- Compare 8 and 4 → swap → [3, 5, 4, 8].
- Keep repeating until sorted → [3, 4, 5, 8].

2. Selection Sort

- **Idea:** Find the smallest element and put it in front.
- **Example:** [29, 10, 14, 37].
 - Smallest is 10 → swap with first → [10, 29, 14, 37].
 - Next smallest is 14 → swap with second → [10, 14, 29, 37].
 - Continue until sorted.

3. Merge Sort

- **Idea:** Divide list into halves, sort each half, then merge.
- **Example:** [38, 27, 43, 3].
 - Split → [38, 27] and [43, 3].
 - Sort each → [27, 38] and [3, 43].
 - Merge → [3, 27, 38, 43].

4. Quick Sort

- **Idea:** Pick a pivot, partition into smaller and larger, then recurse.
- **Example:** [7, 2, 1, 6].
 - Pivot = 7 → left [2, 1, 6], right [].
 - Sort left → [1, 2, 6].
 - Combine → [1, 2, 6, 7].

Searching Algorithms

5. Linear Search

- **Idea:** Check each element one by one.
- **Example:** Find 8 in [3, 5, 8, 4].
 - Check 3 → not match.
 - Check 5 → not match.
 - Check 8 → found at position 3.

6. Binary Search

- **Idea:** Works only on sorted lists; cut search space in half each time.
- **Example:** Find 27 in [10, 14, 27, 38, 43].
 - Middle = 27 → match found immediately.
 - If target < middle, search left; if > middle, search right.

1. Reinforcement Learning in Practice

- **Agent-Environment Loop:** The agent observes a state, chooses an action, receives a reward, and transitions to a new state. This loop continues across episodes.
- **Exploration vs Exploitation:**
 - *Exploration:* Trying new actions to discover rewards.

Exploitation: Using known actions that yield high rewards.

- Example: A robot deciding whether to try a new path in a maze (exploration) or stick to the shortest known path (exploitation).

2. Q-Learning and DQN

Q-Learning: Tabular method; works well for small state spaces.

- Example: Tic-Tac-Toe board states.

DQN: Uses a neural network to approximate Q-values when the state space is too large (like pixels in Atari games).

- Example: Playing “Breakout” by mapping screen pixels → actions (move paddle left/right).

3. Policy Gradient Methods

- Instead of estimating Q-values, these methods directly adjust the policy parameters.
- **REINFORCE Algorithm:** Updates policy weights based on rewards received.
- Example: Training a chatbot to generate polite responses by rewarding “politeness.”

4. Temporal Difference Learning

- TD methods combine ideas of Monte Carlo (learning from complete episodes) and Dynamic Programming (bootstrapping from estimates).
- Example: Predicting the value of a chess position without simulating the entire game.

5. BFS and DFS in AI

- **BFS:**
 - Finds shortest path in unweighted graphs.
 - Example: Social network “degrees of separation.”
- **DFS:**
 - Useful for exhaustive search.

- Example: Generating all possible moves in a puzzle until solution is found.

6. Neural Networks (Deeper Dive)

Hidden Layers: Capture nonlinear relationships.

- Example: Distinguishing handwritten digits (MNIST dataset).
- **Activation Functions:**
 - *Sigmoid*: Good for binary classification.
 - *Softmax*: Good for multi-class classification.
 - *ReLU*: Common in deep networks for faster training.

Backpropagation: Algorithm to update weights by propagating error backward.

- Example: Training a network to recognize cats vs dogs.

7. Backtracking

- Systematically explores possible solutions and abandons paths that fail.
- Example: Solving the N-Queens problem or generating valid Sudoku solutions.

8. Connecting RL with Neural Networks

- DQNs use **neural networks** to approximate Q-values.
- Policy gradient methods use **softmax** for action probabilities.
- Search algorithms (BFS/DFS) can be combined with RL for planning.
- Backtracking is useful in constraint-based environments (like puzzle-solving agents).

9. Example Walkthrough

Imagine training an agent to classify fruits:

1. Input: APPLE
2. Response: MONGO
3. Feedback: Wrong (reward = -1)
4. Learns: Updates Q-values or policy weights
5. Next time: Predicts APPLE correctly (reward = +1)

This cycle illustrates reinforcement learning with feedback.

Trees, BFS, and DFS

What is a Tree?

- A **tree** is a hierarchical data structure consisting of nodes.
- Each node has:
 - **Value** (data)
 - **Children** (links to other nodes)
- The **root** is the top-most node.

A

/ \

B C

/ \

D E

Breadth-First Search (BFS)

- **Strategy:** Explore level by level.
- Uses a **queue** (FIFO).
- Example order for the tree above: A → B → C → D → E.

3. Depth-First Search (DFS)

- **Strategy:** Explore as far as possible down one branch before backtracking.
- Uses a **stack** (or recursion).
- Example orders:
 - Preorder (Root → Left → Right): A → B → D → E → C
 - Inorder (Left → Root → Right): D → B → E → A → C
 - Postorder (Left → Right → Root): D → E → B → C → A

```
from collections import deque
```

```
# Simple BFS on a graph
```

```
def bfs(graph, start):
```

```
    visited = set()
```

```
    queue = deque([start])
```



```
order = []
```

```
while queue:
```

```
    node = queue.popleft()
```

```
    if node not in visited:
```

```
        visited.add(node)
```

```
        order.append(node)
```

```
        # Add all neighbors to the queue
```

```
        for neighbor in graph[node]:
```

```
            if neighbor not in visited:
```

```
                queue.append(neighbor)
```

```
return order
```

```
# Example graph (Adjacency List)
```

```
graph = {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['D', 'E'],
```

```
    'C': ['F'],
```

```
    'D': [],
```

```
    'E': ['F'],
```

```
    'F': []
```

```
}
```

```
print("BFS starting from A:", bfs(graph, 'A'))
```

```
BFS starting from A: ['A', 'B', 'C', 'D', 'E', 'F']
```

AI Algorithms & Techniques Overview

1. Temporal Difference (TD) Learning

- **Definition:** A reinforcement learning method that updates value estimates based on differences between successive predictions.
- **Formula (Bellman's Equation):**

$$Q(s,a)=R(s,a)+\gamma \max_{a'} Q(s',a')$$

- **Key Point:** Combines Monte Carlo and dynamic programming ideas.
- **Applications:** Q-learning, SARSA, game-playing agents.

2. Breadth First Search (BFS)

- **Definition:** Explores nodes level by level from the start node.
- **Data Structure:** Queue.
- **Key Point:** Guarantees shortest path in unweighted graphs.
- **Applications:** Pathfinding, web crawlers, social network analysis.

3. Depth First Search (DFS)

- **Definition:** Explores as deep as possible along one branch before backtracking.
- **Data Structure:** Stack or recursion.
- **Key Point:** Memory-efficient compared to BFS.
- **Applications:** Topological sorting, cycle detection.

4. Depth Limited Search (DLS)

- **Definition:** DFS with a maximum depth limit.
- **Key Point:** Prevents infinite exploration in large search spaces.
- **Applications:** Game trees, bounded search problems.

5. Uniform Cost Search (UCS)

- **Definition:** Expands the node with the lowest path cost first.
- **Data Structure:** Priority queue.
- **Key Point:** Guarantees optimal solution if costs are positive.

- **Applications:** Route planning, weighted graph search.

6. Traveling Salesman Problem (TSP)

- **Definition:** Find the shortest route visiting all cities once and returning to start.
- **Key Point:** NP-hard optimization problem.
- **Applications:** Logistics, circuit design, robotics.

7. Game Playing (Minimax)

- **Definition:** Algorithm for adversarial games assuming optimal opponent play.
- **Key Point:** Often combined with Alpha-Beta pruning to reduce computation.
- **Applications:** Chess, Tic-Tac-Toe, strategy games.

8. Face Search (Recognition)

- **Definition:** Identifying/verifying individuals by comparing facial features.
- **Key Point:** Uses deep learning (CNNs).
- **Applications:** Security systems, social media tagging.

9. Principal Component Analysis (PCA)

- **Definition:** Dimensionality reduction technique.
- **Key Point:** Transforms data into fewer components while preserving variance.
- **Applications:** Data compression, visualization, preprocessing for ML.

10. VGG-16

- **Definition:** A CNN with 16 layers, known for simplicity and effectiveness.
- **Key Point:** Uses small (3×3) convolution filters.
- **Applications:** Image classification, transfer learning.

11. Convolutional Neural Network (CNN)

- **Definition:** Deep learning model for spatial data (images/videos).
- **Key Point:** Automatically extracts hierarchical features.
- **Applications:** Object detection, medical imaging, video analysis.

12. Recurrent Neural Network (RNN)

- **Definition:** Neural network for sequential data with feedback loops.
- **Key Point:** Captures temporal dependencies.
- **Applications:** Language modeling, time series forecasting.

13. Long Short-Term Memory (LSTM)

- **Definition:** A type of RNN that solves vanishing gradient problem.
- **Key Point:** Learns long-term dependencies using gates (input, forget, output).
- **Applications:** Speech recognition, text generation, translation.

14. Bidirectional Search

- **Definition:** Runs two searches simultaneously (from start and goal).
- **Key Point:** Reduces search time compared to one-directional search.
- **Applications:** Pathfinding in large graphs.

15. Hill Climbing Algorithm

- **Definition:** Local search algorithm that moves toward
- **Key Point:** Stops when no improvement is possible (risk of local maxima).
- **Applications:** Optimization problems, AI planning.

Gradient Descent Explained

What It Is

- **Definition:** Gradient Descent is a **first-order iterative optimization algorithm** used to minimize a differentiable function.
- **Intuition:** Imagine standing on a hill and wanting to reach the lowest point. You look at the slope (gradient) and take a step downhill. Repeat until you reach the valley.
- **Mathematical Update Rule:**

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \nabla J(\theta)$$

- θ : model parameters (weights, biases)
- α : learning rate (step size)
- $\nabla J(\theta)$: gradient of the cost function

Types of Gradient Descent

Type	Description	Pros	Cons
------	-------------	------	------

Batch Gradient Descent	Uses the entire dataset to compute gradient	Stable convergence	Very slow for large datasets
Stochastic Gradient Descent (SGD)	Uses one sample at a time	Fast updates, handles large data	Noisy convergence
Mini-Batch Gradient Descent	Uses small batches of data	Balance between speed & stability	Requires tuning batch size

Key Concepts

- **Learning Rate (α):**
 - Too high → overshoot minimum, unstable.
 - Too low → very slow convergence.
- **Cost Function:** Measures error between predicted and actual values (e.g., Mean Squared Error).
- **Convergence:** Algorithm stops when gradient is close to zero or cost function stops decreasing.

Applications

- **Linear Regression:** Finds best-fit line by minimizing squared error.
- **Logistic Regression:** Optimizes classification boundaries.
- **Neural Networks:** Used with backpropagation to update weights.
- **Support Vector Machines (SVMs):** Optimizes separating hyperplane.

Challenges

- **Local Minima:** May get stuck in non-optimal valleys.
- **Saddle Points:** Flat regions slow down learning.
- **Vanishing/Exploding Gradients:** Common in deep networks (solved with LSTM, ReLU, etc.).
- **Choosing Learning Rate:** Critical for performance; adaptive methods like Adam, RMSProp help.

Variants & Improvements

- **Momentum:** Adds inertia to updates, helps escape local minima.

- **AdaGrad:** Adjusts learning rate based on parameter updates.
- **RMSProp:** Normalizes gradients to avoid exploding/vanishing.
- **Adam:** Combines Momentum + RMSProp, widely used in deep learning.

Quick Example

Suppose we want to minimize:

$$J(\theta) = (\theta - 3)^2$$

- Gradient: $\nabla J(\theta) = 2(\theta - 3)$
- Update: $\theta_{\text{new}} = \theta_{\text{old}} - \alpha 2(\theta - 3)$